Loops and Control Flow in Python

Contents

1 Introduction

Till now, our programs were running line by line.

Like obedient students.

But real programs need repetition.

Examples:

- Print welcome message 100 times
- Check password repeatedly
- Generate multiplication tables
- Process thousands of users

Imagine writing:

```
print("Welcome")
print("Welcome")
print("Welcome")
print("Welcome")
```

1000 times.

At some point even your keyboard will protest.

That is why loops exist.

2 Control Flow

Control flow means:

How program execution moves.

Two major parts:

- Conditional Statements
- Looping Statements

We already learned conditional statements using:

- if
- else
- elif

Now we learn loops.

3 Looping Statements

Loops repeat code multiple times.

Mainly two types:

- Finite Looping
- Infinite Looping

4 Finite Looping

Finite means:

Loop stops after fixed number of repetitions.

Usually done using:

```
for loop
```

5 Problem Without Loop

```
print("Welcome home !!!")
print("Welcome home !!!")
print("Welcome home !!!")
print("Welcome home !!!")
print("Welcome home !!!")
```

Works.
But not smart.
Programmers are lazy in a productive way.
We prefer:

Write less, do more.

6 For Loop

```
for i in range(5):
    print("Welcome home !!!")
```

Output:

```
Welcome home !!!
Welcome home !!!
Welcome home !!!
Welcome home !!!
Welcome home !!!
```

Much cleaner.

7 Understanding the Loop Variable

7.1 What is `i`?

`i` is called loop variable.

Example:

```
for i in range(5):
    print(i)
```

Output:

```
0
1
2
3
4
```

8 Understanding Range

8.1 `range(5)`

```
range(5)
```

acts like:

```
[0, 1, 2, 3, 4]
```

Notice:

- Starts from 0
- Stops before 5

Python says:

"I will go till 5... but not actually touch 5."

9 Range with Start and Stop

```
for i in range(1, 5):
    print(i)
```

Output:

```
1
2
3
4
```

Meaning:

- 1 = starting value
- 5 = stopping value
- stopping value excluded

10 Range with Step Value

```
for i in range(1, 5, 2):
    print(i)
```

Output:

```
1
3
```

Meaning:

- Start from 1
- Stop before 5
- Jump by 2

That last number is called:

Step Value

11 Multiplication Table Problem

Without loops:

```
print("2x1=2")
print("2x2=4")
print("2x3=6")
...
```

This becomes boring very quickly.
So we use loops.

```
for num in range(1, 11):
    print("2 x", num, "=", num*2)
```

Output:

```
2 x 1 = 2
2 x 2 = 4
...
2 x 10 = 20
```

12 F Strings

Python gives cleaner formatting using:

```
f""
```

Example:

```
for num in range(1, 11):
    print(f"3 x {num} = {3*num}")
```

Inside:

```
{}
```

we can directly write variables or calculations.
This is called:

Formatted String

or simply:

F-string

13 Dynamic Multiplication Table

Now user chooses number.

```
number = int(input("Enter a number for mul table: "))

for num in range(1, 11):
    print(f"{number} x {num} = {number*num}")
```

Example Input:

```
5
```

Output:

```
5 x 1 = 5
5 x 2 = 10
...
5 x 10 = 50
```

14 Reverse Looping

Loops can also move backwards.

```
for num in range(10, 0, -1):
    print(num)
```

Output:

```
10
9
8
...
1
```

Notice:

- Negative step value
- Loop decreases

15 Strings are Sequences

```
sentence = "We love Nepal"
```

A string is sequence of characters.
Meaning Python internally sees it somewhat like this:

```
W
e

l
o
v
e

N
e
p
a
l
```

Each character has its own position.
Python allows us to loop through strings character by character.

15.1 Example

```
sentence = "We love Nepal"

for each_char in sentence:
    print(each_char)
```

Output:

```
W
e

l
o
v
e

N
e
p
a
l
```

Here:

```
each_char
```

stores one character at a time.
First:

```
W
```

Then:

```
e
```

Then space, then next characters and so on.

16 Membership Operator

```
in
```

is called:

Membership Operator

It checks whether something exists inside another thing.

16.1 Example 1

```
print('e' in "aeiou")
```

Output:

```
True
```

Why?
Because:

```
e
```

exists inside:

```
aeiou
```

16.2 Example 2

```
print('W' in "aeiou")
```

Output:

```
False
```

Because:

```
W
```

does not exist inside:

```
aeiou
```

16.3 Real Example with Loop

```
sentence = "We love Nepal"
vowels = "aeiouAEIOU"

for each_char in sentence:

    if each_char in vowels:
        print(each_char)
```

Output:

```
e
o
e
e
a
```

Program checks:

- Is W inside vowels? No
- Is e inside vowels? Yes
- Is space inside vowels? No
- Is o inside vowels? Yes

and so on.
This is how we can search characters inside strings.

17 Finding Vowels in Sentence

```
sentence = "We love Nepal"
vowels = "aeiouAEIOU"

for each_char in sentence:
    if each_char in vowels:
        print(each_char)
```

Output:

```
e
o
e
e
a
```

Program checks each character one by one.

18 Counting Vowels

```
sentence = "We love Nepal. We speak Nepali"
vowels = "aeiouAEIOU"

count = 0

for each_char in sentence:
    if each_char in vowels:
        print(each_char)
        count += 1

print("No of vowels in sentence is", count)
```

19 What is +=?

```
count += 1
```

means:

```
count = count + 1
```

Used for increasing value repeatedly.
Very common in loops.

20 Underscore Variable

Sometimes we do not need loop variable.
Example:

```
for _ in range(5):
    print("Welcome home !!!")
```

Here:

means:

"I do not care about loop variable."

Python programmers use underscore for ignored values.

21 Infinite Looping

Infinite means:

Loop continues forever until stopped.

Usually done using:

```
while loop
```

22 While Loop

Example:

```
line_xa = True

while line_xa:
    print("Fan ghumdaixa")
```

This loop never stops.
Because:

```
line_xa
```

always remains:

```
True
```

23 Stopping Infinite Loop

```
line_xa = True

while line_xa:
    print("Fan ghumdaixa")

    user_answer = input("Line xa ?? (yes/no): ")

    if user_answer == "no":
        line_xa = False
```

Now loop stops when:

```
line_xa = False
```

24 Real World Example: Account Blocking System

```
dummy_password = "admin123"
MAX_ATTEMPTS = 3
user_attempt = 0
```

Explanation:

- Correct password stored
- Maximum attempts fixed
- Current attempt count stored

25 While Loop Condition

```
while user_attempt < MAX_ATTEMPTS:
```

Meaning:

Keep asking password while attempts are less than maximum allowed attempts.

26 Password Checking

```
if user_password != dummy_password:
```

Meaning:

If password does not match.

27 Increasing Attempt Count

```
user_attempt += 1
```

Each wrong password increases attempts.

28 Break Statement

```
break
```

immediately stops loop.
Example:

```
print("Login successful")
break
```

As soon as correct password entered:

- Loop ends
- Program continues below loop

29 Continue Statement

```
continue
```

means:

Skip remaining code and jump to next loop iteration.

Example:

```
if user_password != dummy_password:
    print("Wrong password !!")
    continue
```

Everything below continue inside current loop iteration gets skipped.

30 Final Program

```
dummy_password = "admin123"
MAX_ATTEMPTS = 3
user_attempt = 0

while user_attempt < MAX_ATTEMPTS:
    user_password = input("Enter your password: ")

    if user_password != dummy_password:
        print("Wrong password !!")
        user_attempt += 1
        print("Attempts left: ", MAX_ATTEMPTS-user_attempt)
        continue

    print("Login successful")
    break

print("Program end")
```

31 Program Flow Understanding

Suppose:

- Correct password = admin123

User enters:

```
hello
```

Result:

- Wrong password printed
- Attempts increase
- continue runs
- Loop repeats

Finally user enters:

```
admin123
```

Then:

- Login successful printed
- break runs
- Loop stops

32 Final Thoughts

Today you learned:

- Control flow
- for loop
- range()
- Step values
- Reverse looping
- Membership operator
- String traversal
- Counting
- while loop
- Infinite looping
- break
- continue

Now your programs can:

Repeat tasks automatically.

Congratulations.
You officially made computer do repetitive work instead of yourself.
True programmer energy.

33 Exercises

33.1 Exercise 1: Print Numbers

Use loop to print numbers from:

- 1 to 10

33.2 Exercise 2: Reverse Numbers

Print numbers from:

- 10 to 1

33.3 Exercise 3: Multiplication Table

Take number input from user and print multiplication table till 10.

33.4 Exercise 4: Sum of Numbers

Find sum of numbers from 1 to 100.

Hint:

```
total += num
```

33.5 Exercise 5: Count Vowels

Take sentence input from user and count vowels.

33.6 Exercise 6: Find Even Numbers

Print all even numbers between 1 and 50.

Hint:

```
num % 2 == 0
```

33.7 Exercise 7: Password System

Create login system with:

- Maximum 3 attempts
- Correct password check
- Account blocked message

33.8 Exercise 8: Countdown Timer

Print countdown:

```
10
9
8
...
1
Blast off!
```

33.9 Exercise 9: Character Counter

Take sentence input and count:

- Total characters

33.10 Exercise 10: Challenge Exercise

Take sentence input and print:

- Number of vowels
- Number of consonants

34 Homework Advice

Do not only read loops.
Run them.
Break them.
Modify them.
Loops become easy only after writing many loops.
At first loops feel confusing.
Then suddenly one day:

"Ohhhh... that's all it was?"

That day is near.